

Threat Intelligence Platform Architecture Design

THE ARZENS — Advanced Track, Week 06 — Task 1

Prepared by Saqlain — AI, Automation & Security Engineering Track

1. Introduction

Modern Security Operations Centers (SOCs) are overwhelmed by raw indicators — IPs, domains, file hashes, and URLs — surfaced by firewalls, EDR agents, and SIEM correlation rules. On their own, these indicators carry no context: an analyst cannot tell whether **185.220.101.45** is a Tor exit node, a known command-and-control server, or entirely benign. A Threat Intelligence (TI) enrichment architecture closes this gap by automatically cross-referencing every observed indicator against multiple reputation sources, assigning it a risk score, and feeding that context back into the SOC's existing workflow. This document describes the architecture built for THE ARZENS internship Task 2/3 deliverables: a Python enrichment engine (**ti_enricher.py**) backed by an IOC lifecycle manager (**ioc_manager.py**).

2. Architecture Overview

The platform is organized as a five-stage pipeline: indicator ingestion, multi-source enrichment, IOC storage with lifecycle management, workflow integration, and analyst feedback. Each stage is a discrete, independently testable component, which keeps the system maintainable and lets any single source (e.g. VirusTotal) be swapped or extended without touching the rest of the pipeline. Figure 1 illustrates the full data flow.

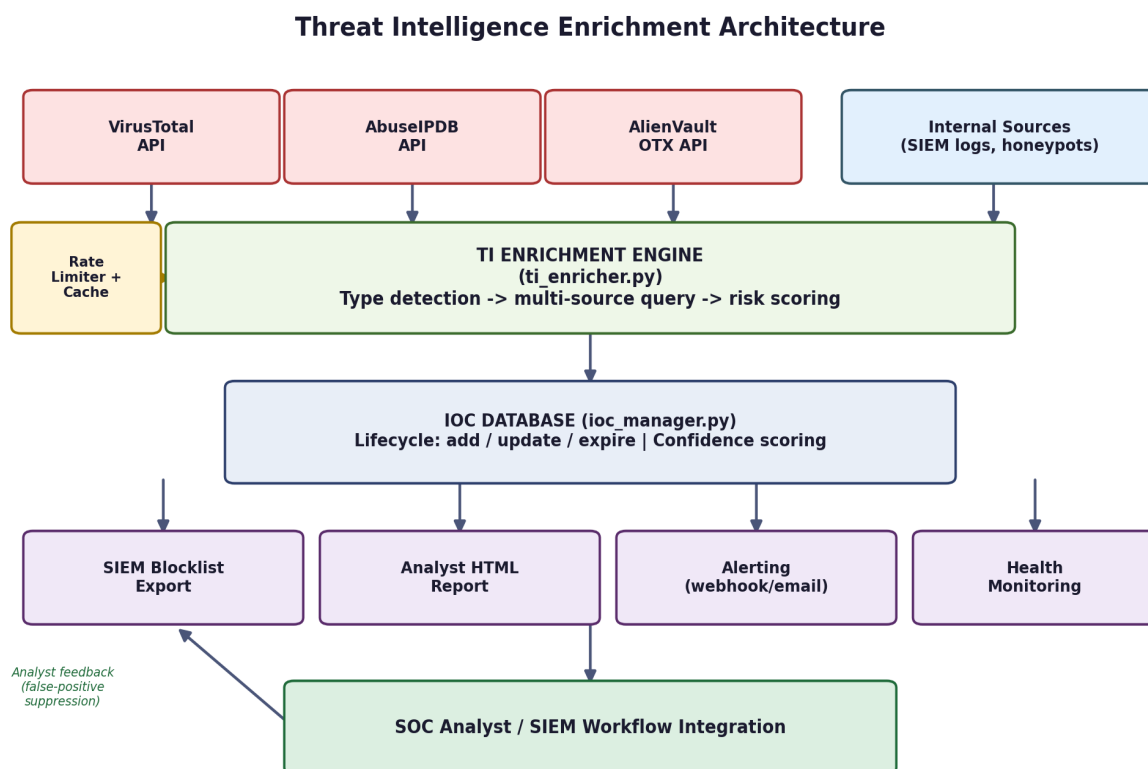


Figure 1 — Threat Intelligence enrichment architecture and data flow.

3. Data Sources

3.1 External sources

Three free-tier external TI feeds are queried per indicator: **VirusTotal** (multi-engine malware/URL/IP reputation), **AbuseIPDB** (community-reported IP abuse confidence, IPs only), and **AlienVault OTX** (pulse-based threat-actor and campaign correlation). Querying three independent sources rather than one avoids single-vendor blind spots and lets the risk-scoring stage cross-validate a verdict before it reaches an analyst.

3.2 Internal sources

Internally, the architecture is designed to also ingest indicators surfaced by the organization's own SIEM correlation rules and any deployed honeypots. These internal observations are treated as a fourth signal in the scoring model going forward and, critically, provide organization-specific context (e.g. "this IP has hit our honeypot 40 times this week") that no external feed can offer.

4. Enrichment Engine

The enrichment engine first classifies each indicator by type — IP, domain, URL, or file hash (MD5/SHA1/SHA256, detected by length and character set) — since each TI source exposes a different

API endpoint per type (AbuseIPDB, for instance, only accepts IPs). It then queries all applicable sources in sequence, respecting each source's documented rate limit through a sliding-window limiter, and caches every successful response for 24 hours so repeated lookups on the same indicator do not exhaust the free-tier quota. Each source returns its own 0-100 score; these are combined into a single weighted risk score (VirusTotal 50%, AbuseIPDB 30%, OTX 20%). If a source fails or does not apply to the indicator type, its weight is redistributed across the sources that did respond, rather than silently dragging the score toward zero.

5. IOC Database

Enriched indicators are persisted in a JSON-backed IOC database keyed by indicator value, storing **first_seen**, **last_seen**, a verdict-based **expiration** date, and a **confidence** score computed from three rule-based factors: source diversity (how many of the three feeds contributed a signal), score stability (variance across historical re-enrichments — a consistent verdict is trusted more than a one-off spike), and recency (an exponential decay since the indicator was last confirmed). This confidence score lets an analyst distinguish a freshly-flagged, well-corroborated MALICIOUS indicator from a stale, single-source hit.

6. Workflow Integration

The IOC database drives four downstream integrations: a plaintext or firewall-formatted **SIEM blocklist** export for automatic ingestion into perimeter controls; a styled **HTML analyst report** summarizing verdict counts and top-risk indicators for the weekly SOC review; a stubbed **alerting** hook that can post to Slack/Teams/PagerDuty for newly-confirmed MALICIOUS indicators; and a **health-check** command exposing database size, average confidence, and staleness metrics for operational monitoring.

7. Rate Limit Handling

Each source's free-tier quota is enforced client-side by a sliding-window rate limiter that sleeps the calling thread rather than firing a request that would return HTTP 429. This is combined with the 24-hour result cache, so a batch of a thousand indicators that includes repeats only spends rate-limit budget once per unique indicator per day — critical given VirusTotal's public tier allows only four requests per minute.

8. IOC Expiration

IOCs are not retained indefinitely: **expiration_days** is keyed by verdict in configuration, so a MALICIOUS indicator is retained for 90 days (valuable for retrospective threat hunting) while a CLEAN indicator ages out after just 3 days. A scheduled **--expire-check** pass removes anything past its expiry, keeping the database — and any exported blocklist — lean and preventing stale, no-longer-relevant entries from generating false blocks.

9. False Positive Prevention

False positives are mitigated at three layers: (1) the weighted multi-source scoring means a single overzealous source cannot alone push an indicator to MALICIOUS; (2) the confidence score surfaces

low-diversity or unstable verdicts so analysts can triage them separately from high-confidence hits; and (3) an explicit analyst feedback loop (shown in Figure 1) lets a human override or suppress a verdict, with that decision fed back into future scoring runs. Together, these layers ensure automation accelerates triage without removing human judgment from high-impact blocking decisions.

10. Reference Architecture

This design deliberately mirrors the ingestion/enrichment/storage separation used by the open-source **MISP Project** (misp-project.org), the de-facto standard for threat intelligence platforms, while remaining lightweight enough to run as two standalone Python scripts rather than a full MISP deployment — appropriate for the scope of this internship task while following the same architectural principles a production SOC platform would use.

11. Conclusion

This architecture turns raw, context-free indicators into scored, confidence-rated intelligence that a SOC can act on directly — automatically respecting API constraints, aging out stale data, and leaving room for analyst oversight at every stage. The corresponding implementation (**ti_enricher.py** and **ioc_manager.py**) is submitted alongside this document as Tasks 2 and 3.